

GRAVEATLAS

Phase 6 — Security, Privacy, Trust, Safety & Operational Hardening

Full copy-paste implementation prompt for the Base44 agent.

COPY-PASTE MASTER PROMPT

GRAVEATLAS – PHASE 6
SECURITY, PRIVACY, TRUST, SAFETY & OPERATIONAL HARDENING

Phase 1 through Phase 5 must already be complete.

Continue from the existing GraveAtlas implementation.

IMPORTANT:

- Do NOT rebuild Phases 1-5.
- Reuse the existing Android app, Cloudflare Worker/API, GitHub App, public data repository, schemas, search, map, authentication, etc.
- Do NOT start Phase 7.
- Do NOT add paid services without explicit approval.
- Do NOT expose secrets.
- Do NOT weaken existing security controls.
- Do NOT fabricate security test results.
- Do NOT delete legitimate cemetery or historical data.
- Do NOT perform destructive production operations.
- Fix security issues at the smallest practical scope.
- Complete only Phase 6, then stop.

=====

PART 1 – SECURITY INVENTORY

=====

Create a current security inventory covering:

- Android application
- Cloudflare Worker/API
- GitHub App
- GitHub repository
- authentication
- authorization
- database/data store
- public APIs
- contribution system
- moderation system
- publication pipeline
- maps/location features
- caching
- logs
- CI/CD
- secrets

Classify each component:

LOW
MEDIUM
HIGH
CRITICAL

Only use severity classifications based on actual findings.

=====

PART 2 – SECRET INVENTORY

=====

Identify where secrets are expected to exist.

Examples:

GITHUB_APP_ID
GITHUB_PRIVATE_KEY
GITHUB_INSTALLATION_ID
ADMIN_TOKEN
Cloudflare credentials
authentication secrets
API keys

Do NOT print secret values.

Report only:

- variable name
- storage location
- consuming component
- whether exposed risk exists

=====

PART 3 – ANDROID SECRET SCAN

=====

Inspect Android source/configuration for:

- private keys
- API tokens
- passwords
- admin tokens
- GitHub credentials
- embedded credentials
- unsafe debug secrets

Remove or replace unsafe values.

Do not commit real secrets.

=====

PART 4 – BACKEND SECRET SCAN

=====

Inspect backend source and configuration.

Verify secrets are loaded securely through:

- environment variables
- GitHub Actions secrets
- Cloudflare secret storage
- other existing secure mechanisms

Never expose secret values in logs or API responses.

=====

PART 5 – GITHUB SECURITY

=====

Review GitHub App configuration and repository permissions.

Verify:

- least privilege
- installation scope
- repository access
- workflow permissions
- branch protections where applicable
- secret protection
- pull request security

Do not request broad permissions unnecessarily.

=====

PART 6 – GITHUB ACTIONS SECURITY

=====

Review workflows for:

- excessive permissions
- untrusted pull-request execution
- secret exposure
- unsafe shell interpolation
- third-party action risks
- artifact exposure
- dependency risks

Pin or constrain actions where practical without breaking the workflow.

=====

PART 7 – AUTHENTICATION REVIEW

=====

Review:

- login
- logout
- sessions
- expiration
- refresh
- unauthorized access
- account state

Ensure authentication cannot be bypassed through client-side manipulation.

=====

PART 8 – AUTHORIZATION REVIEW

=====

Test server-side enforcement for:

PUBLIC
CONTRIBUTOR
MODERATOR
ADMIN

Attempt safe unauthorized access tests.

Verify users cannot:

- access another user's private data
- moderate without permission
- publish without permission
- execute admin operations
- access private moderation notes

=====

PART 9 – IDOR / OBJECT ACCESS REVIEW

=====

Review endpoints using IDs such as:

- contribution ID
- correction ID
- user ID
- moderation ID
- publication ID

Verify changing an identifier cannot expose another user's private resource.

=====

PART 10 – INPUT VALIDATION

=====

Review all user-controlled inputs.

Validate:

- strings
- IDs
- dates
- coordinates
- filenames
- URLs
- JSON
- query parameters
- request bodies

Reject malformed input safely.

=====

PART 11 – INJECTION PROTECTION

=====

Review protection against:

- SQL/NoSQL injection where applicable
- command injection
- path traversal
- template injection
- JSON manipulation
- header injection
- repository path manipulation

Do not execute untrusted input as commands.

=====
PART 12 – XSS / CONTENT SAFETY
=====

Review user-submitted text and public content.

Ensure content cannot execute arbitrary scripts in web/admin surfaces.

Escape or sanitize where appropriate.

Do not render untrusted HTML without controlled sanitization.

=====
PART 13 – FILE / MEDIA SECURITY
=====

If uploads exist:

verify:

- file size limits
- MIME validation
- extension validation
- filename sanitization
- storage isolation
- access control
- executable-file prevention

If uploads are not implemented:

document:

NOT IMPLEMENTED

=====
PART 14 – RATE LIMITING
=====

Review rate limits for:

- authentication
- search
- API access
- contribution submission
- correction submission
- moderation
- publication
- administrative operations

Prevent abuse without blocking normal users unnecessarily.

=====
PART 15 – ABUSE PROTECTION
=====

Identify potential abuse patterns:

- automated submissions
- duplicate contributions
- spam
- scraping
- brute force
- repeated failed authentication
- publication abuse
- map/API query abuse

Implement practical protections using existing infrastructure.

=====
PART 16 – PRIVACY DATA INVENTORY
=====

Document data collected by GraveAtlas.

Classify:

PUBLIC
PRIVATE

INTERNAL
SECURITY-SENSITIVE

Minimize collection.

Do not collect information merely because it might be useful later.

=====
PART 17 – LOCATION PRIVACY
=====

Review user location handling.

Verify:

- permission is optional where practical
- precise location is not unnecessarily retained
- location is not accidentally logged
- location is not exposed through API responses
- nearby search uses minimum necessary information

=====
PART 18 – CONTRIBUTOR PRIVACY
=====

Review contributor information.

Public records should not unnecessarily reveal:

- private email
- authentication identifiers
- internal user IDs
- private moderation information
- private notes

Only expose contributor attribution when intentionally designed as public.

=====
PART 19 – MODERATOR PRIVACY
=====

Ensure moderation activity does not unintentionally expose moderator identities or private notes to ordinary users.

Keep internal moderation data protected.

=====
PART 20 – DATA RETENTION
=====

Document retention behavior for:

- accounts
- sessions
- contributions
- corrections
- moderation records
- audit records
- logs
- cached data

Do not implement arbitrary deletion of historical/public records.

=====
PART 21 – DATA DELETION / CORRECTION
=====

Verify appropriate mechanisms for user-controlled private data.

Distinguish:

PERSONAL ACCOUNT DATA
PUBLIC HISTORICAL DATA
AUDIT RECORDS
MODERATION RECORDS

Do not erase audit history merely because a public record is corrected.

=====
PART 22 – LOGGING REVIEW
=====

Inspect logs for accidental exposure of:

- passwords
- tokens
- private keys
- authentication headers
- private user data
- moderation notes
- precise location

Redact sensitive values.

=====

PART 23 – ERROR RESPONSE REVIEW

=====

Verify API errors do not expose:

- stack traces
- filesystem paths
- database details
- internal credentials
- infrastructure secrets

Return safe error messages with correlation/request IDs where appropriate.

=====

PART 24 – SECURITY HEADERS / TRANSPORT

=====

Where applicable, verify secure transport and security headers.

Do not claim HTTPS or security-header protection unless actually configured.

=====

PART 25 – CORS REVIEW

=====

Review CORS.

Allow only the origins required by the actual application architecture.

Do not use unrestricted wildcard configuration for authenticated privileged operations unless clearly safe and justified.

=====

PART 26 – CACHE SECURITY

=====

Verify private responses cannot enter public caches.

Review:

- authentication responses
- contributor data
- moderation data
- admin responses
- public cemetery data

Ensure cache keys cannot mix users or permissions.

=====

PART 27 – PUBLIC DATA INTEGRITY

=====

Verify public data cannot be modified by:

- ordinary users
- unauthorized API calls
- manipulated contribution requests
- forged GitHub operations

All publication must pass the Phase 4 controlled pipeline.

=====

PART 28 – AUDIT INTEGRITY

=====

Verify important security events are auditable.

Include where applicable:

- authentication events
- privileged actions
- moderation decisions
- publication operations
- security-sensitive changes

Do not log secrets.

=====

PART 29 – DEPENDENCY SECURITY

=====

Review project dependencies.

Identify:

- outdated critical dependencies
- known security risks
- unnecessary dependencies
- abandoned packages

Do not blindly upgrade everything.

Use compatible upgrades only.

=====

PART 30 – MOBILE SECURITY

=====

Review Android configuration for:

- debug builds
- release signing
- exported components
- deep links
- WebViews if present
- insecure storage
- backup behavior
- network security configuration

Do not break required application functionality.

=====

PART 31 – DEEP LINK SECURITY

=====

If deep links/app links exist:

verify:

- allowed schemes/domains
- parameter validation
- authentication boundaries
- malicious URL handling

Do not allow deep links to bypass authorization.

=====

PART 32 – SECURITY TESTING

=====

Run safe tests for:

- authentication bypass
- authorization bypass
- IDOR
- input validation
- injection
- path traversal
- rate limits
- secret exposure
- privacy leakage
- cache leakage
- GitHub permission abuse
- publication abuse

Do not perform destructive exploitation.

=====

PART 33 – BACKUP / RECOVERY SECURITY

=====

Review backup handling.

Verify backups do not expose:

- secrets
- private contributor data
- credentials

Document access controls and recovery limitations.

=====

PART 34 – INCIDENT RESPONSE

=====

Create/update:

docs/SECURITY-INCIDENT-RESPONSE.md

Include procedures for:

- leaked credential
- compromised GitHub App
- unauthorized publication
- account compromise
- data exposure
- malicious contribution
- API abuse

Do not include real secrets.

=====

PART 35 – SECURITY DOCUMENTATION

=====

Create/update:

docs/SECURITY-AUDIT.md
docs/SECRETS-MANAGEMENT.md
docs/PRIVACY-DATA-MAP.md
docs/ACCESS-CONTROL.md
docs/ABUSE-PREVENTION.md
docs/INCIDENT-RESPONSE.md

Document actual implementation only.

=====

PHASE 6 ACCEPTANCE GATE

=====

Verify:

- [] Security inventory
- [] Secret inventory
- [] Android secret scan
- [] Backend secret scan
- [] GitHub security
- [] GitHub Actions security
- [] Authentication review
- [] Authorization review
- [] IDOR review
- [] Input validation
- [] Injection protection
- [] XSS/content safety
- [] File/media security
- [] Rate limiting
- [] Abuse protection
- [] Privacy data inventory
- [] Location privacy
- [] Contributor privacy
- [] Moderator privacy
- [] Data retention
- [] Data deletion/correction
- [] Logging review
- [] Error response security
- [] Transport/security headers
- [] CORS
- [] Cache security
- [] Public data integrity

[] Audit integrity
[] Dependency security
[] Mobile security
[] Deep-link security
[] Security testing
[] Backup/recovery security
[] Incident response
[] Documentation

=====

FINAL REPORT

=====

Return:

PHASE 6 STATUS:
READY / NOT READY

SECRETS:
PASS / FAIL

AUTHENTICATION:
PASS / FAIL

AUTHORIZATION:
PASS / FAIL

API SECURITY:
PASS / FAIL

ANDROID SECURITY:
PASS / FAIL

GITHUB SECURITY:
PASS / FAIL

PRIVACY:
PASS / FAIL

LOCATION PRIVACY:
PASS / FAIL

ABUSE PROTECTION:
PASS / FAIL

DATA INTEGRITY:
PASS / FAIL

AUDIT:
PASS / FAIL

DEPENDENCIES:
PASS / FAIL

INCIDENT RESPONSE:
PASS / FAIL

SECURITY TESTS:
PASS / FAIL

DOCUMENTATION:
PASS / FAIL

=====

ACTUAL SECURITY TEST RESULTS

=====

Report actual results only.

Secret scan:
[actual result]

Authentication:
[actual result]

Authorization:
[actual result]

IDOR:
[actual result]

Input validation:
[actual result]

Injection:
[actual result]

Rate limiting:
[actual result]

Privacy:
[actual result]

Cache isolation:
[actual result]

GitHub security:
[actual result]

Android security:
[actual result]

Do NOT invent results.

=====
CRITICAL FINDINGS
=====

List all:

CRITICAL
HIGH
MEDIUM
LOW

findings.

For each:

- component
- vulnerability/problem
- impact
- evidence
- remediation
- status

Never include secret values.

=====
BLOCKERS
=====

List every issue preventing:

PHASE 6 = READY

=====
NON-BLOCKING ISSUES
=====

List remaining improvements.

=====
STOP CONDITION
=====

Do NOT start Phase 7.

Do NOT add paid security services without explicit approval.

Do NOT expose secrets.

Do NOT perform destructive security testing.

Do NOT fabricate security results.

Do NOT weaken existing authorization.

Do NOT delete legitimate cemetery or historical data.

STOP after the final report.

WAIT FOR EXPLICIT INSTRUCTIONS FOR PHASE 7.